



**AKADEMIA VIZJA
w WARSZAWIE**

**Wydział Technologii informatycznych
Studia stacjonarne**

Przedmiot: Uczenie maszynowe

SPRAWOZDANIE z realizacji projektu laboratoryjnego:

**Klasteryzacja klientów na podstawie danych
Mall_Customers z użyciem KMeans**

Studia stacjonarne, grupa L2

Mikita Kutsayeu 48860

Celem projektu jest opracowanie i zaimplementowanie algorytmu uczenia maszynowego służącego do rozwiązania problemu klasteryzacji danych. Projekt zakłada zarówno teoretyczne uzasadnienie wyboru metody klasteryzacji, jak i praktyczną implementację rozwiązania w języku Python, działającą w środowisku Jupyter Notebook w Google Colaboratory.

Opis wymagań systemu informacyjnego

Zakres działania i funkcjonalności:

- Interaktywny import danych CSV.
- Wstępna analiza danych (rozkłady, korelacje, typy zmiennych).
- Wizualizacja danych liczbowych (Seaborn, Matplotlib).
- Dobór cech do modelu klasteryzacji.
- Normalizacja danych (StandardScaler).
- Automatyczna detekcja optymalnej liczby klastrów (metoda łokcia).
- Trenowanie modelu KMeans.
- Wizualizacja klastrów z centroidami.
- Eksport wyników klasteryzacji (opcjonalnie).

Ograniczenia systemu:

- Działa wyłącznie na danych liczbowych.
- Wymaga formatu CSV.
- Wrażliwość KMeans na wartości odstające.
- Liczba klastrów musi być określona ręcznie lub metodą heurystyczną.

Opis użytego algorytmu

Charakterystyka:

- KMeans – metoda klasteryzacji oparta na minimalizacji sumy odległości euklidesowej od centroidów.
- Nienadzorowane uczenie – brak etykiet w danych.
- Szybkość działania i prostota.

Zalety:

- Dobre działanie przy dużych zbiorach danych liczbowych.
- Intuicyjna interpretacja klastrów.
- Łatwość wizualizacji 2D/3D.

Ograniczenia:

- Wymaga znajomości liczby klastrów.
- Wrażliwość na wartości odstające i skalowanie cech.
- Możliwość zbiegu do lokalnego minimum.

Odniesienia literaturowe:

- Scikit-learn documentation: <https://scikit-learn.org/stable/modules/clustering.html#k-means>
- Bishop C.M., *Pattern Recognition and Machine Learning*

Miary jakości:

- Wykorzystano *inertia* (wewnętrzna suma kwadratów błędów).
- Metoda łokcia do wyboru liczby klastrów.

Szczegółowy opis implementacji

Odwołania do kodu:

- Blok 1: Import bibliotek.

```
[24] # Importujemy niezbędne biblioteki:
import pandas as pd          # do wczytywania i obróbki danych
import numpy as np           # operacje numeryczne
import matplotlib.pyplot as plt # wykresy
import seaborn as sns        # zaawansowana wizualizacja
from google.colab import files
from sklearn.cluster import KMeans # algorytm KMeans
from sklearn.preprocessing import StandardScaler # skalowanie danych
import os
```

- Blok 2: Załadowanie danych + analiza wstępna.

```
[26] # Interaktywny wybór pliku CSV z komputera
uploaded = files.upload()

# Wyświetlanie pierwszych wierszy tabeli
print(df.head())

# Ogólne informacje o danych
print(df.info())
```

- Blok 3: Wizualizacja rozkładów danych.

```
[27] # Zbudujemy wykres par dla cech liczbowych
sns.pairplot(df[['Age', 'Annual Income (k$)', 'Spending Score (1-100)']])
plt.suptitle("Związki między cechami liczbowymi", y=1.02)
plt.show()
```

- Blok 4: Przygotowanie danych (feature selection + standaryzacja).

```
[28] # Wybieramy dwa kluczowe cechy do klasteryzacji
X = df[['Annual Income (k$)', 'Spending Score (1-100)']].copy()

# Standaryzacja cech
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

- Blok 5: Metoda łokcia (inertia + wykres).

```
[29] # Obliczmy metrykę „inertia” dla różnych wartości k
inertia = []
for k in range(1, 11):
    km = KMeans(n_clusters=k, init='k-means++', random_state=42)
    km.fit(X_scaled)
    inertia.append(km.inertia_)

# Wizualizujemy metodę łokcia
plt.figure(figsize=(8, 4))
plt.plot(range(1, 11), inertia, marker='o')
plt.title('Metoda łokcia do wyboru liczby klastrów')
plt.xlabel('Liczba klastrów')
plt.ylabel('Inertia')
plt.grid()
plt.show()
```

- Blok 6: Trenowanie modelu KMeans.

```
[30] # Trenujemy model z 5 klastrami
k = 5
kmeans = KMeans(n_clusters=k, init='k-means++', random_state=42)
y_kmeans = kmeans.fit_predict(X_scaled)

# Dodajemy informacje o przynależności do klastra do ramki danych
df['Cluster'] = y_kmeans
```

- Blok 7: Wizualizacja i przypisanie klastrów.

```
[34] # Wizualizacja klastrów i centroidów
plt.figure(figsize=(8, 6))
colors = ['red', 'blue', 'green', 'purple', 'orange']
for i in range(k):
    plt.scatter(X_scaled[y_kmeans == i, 0], X_scaled[y_kmeans == i, 1],
                s=100, c=colors[i], label=f'Klaster {i}')

# Centroidy
centroids = kmeans.cluster_centers_
plt.scatter(centroids[:, 0], centroids[:, 1], s=300, c='yellow', label='Centroidy', marker='X')

plt.title('Klasyfikacja klientów na podstawie dochodów i wydatków')
plt.xlabel('Dochód (znormalizowany)')
plt.ylabel('Ocena kosztów (znormalizowana)')
plt.legend()
plt.grid()
plt.show()
```

- Blok 8: Interpretacja wyników.

```
[35] # Wyświetlanie pierwszych wierszy z przypisanymi klastrami
print(df[['CustomerID', 'Gender', 'Age', 'Annual Income (k$)', 'Spending Score (1-100)', 'Cluster']])
```

	CustomerID	Gender	Age	Annual Income (k\$)	Spending Score (1-100)	Cluster
0	1	Male	19	15	39	4
1	2	Male	21	15	81	2
2	3	Female	20	16	6	4
3	4	Female	23	16	77	2
4	5	Female	31	17	40	4

Struktura kodu:

- Sekcyjna organizacja kodu.
- Komentarze opisujące działanie każdej komórki.
- Wykorzystanie bibliotek: sklearn, pandas, matplotlib, seaborn.

Instrukcja użytkowania systemu informacyjnego

Jak uruchomić:

- Otwórz plik .ipynb w Google Colab lub Jupyter Notebook.
- Wgraj plik Mall_Customers.csv.
- Wykonuj komórki w kolejności (od góry do dołu).

Przykłady użycia:

- Automatyczne wykrycie liczby klastrów.
- Trening modelu.
- Interaktywna analiza rozkładu klastrów klientów.

Spis literatury

- C.M. Bishop – Pattern Recognition and Machine Learning
- Scikit-learn documentation
- Artykuły i blogi dotyczące KMeans i metody łokcia

Spis załączników

- Projekt_UM_L2_48860.ipynb – notatnik z kodem
- Mall_Customers.csv – dane źródłowe